```
Output:
int(1)
int(2)
```

In the code above, we add a cost of *2* for insertion and replacement and *1* for deletion, which moves "statue" further away from "status" than "statu". A user inputting "statu" to our program is (probably) more likely to have meant to type "status" than if they had typed "statue". This is one of the common uses of Levenshtein's algorithm; for example, the version control system git makes use of it to suggest close matches when sub-commands are incorrectly entered:

```
$ git statu
git: 'statu' is not a git command. See 'git --help'.

The most similar commands are
    status
    stage
    stash
```

Be aware that levenshtein() performs a byte-level comparison, not a character comparison, so multi-byte string comparison will yield larger values than expected, depending on encoding. It should not be considered multi-byte safe, but it is otherwise binary safe.

An alternative algorithm is available via the similar_text() function:

```
$percentRef = 0.0;
var_dump(similar_text(
    "Hello World",
    "hello world",
    $percentRef
));
print "Strings are {$percentRef}% similar". PHP_EOL;

Output:
int(9)
Strings are 81.818181818182% similar
```